

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Make file.

Task -1: Create Project Structure as follows:

```
OSSP/
├── .git/           # Git repository (created automatically)
├── .gitignore      # Files/folders Git should ignore
├── README.md       # Project description and instructions
├── LICENSE         # Project license (optional)
├── Makefile        # Build instructions
├── src/            # Source code
│   ├── main.c
│   ├── shell.c
│   ├── parser.c
│   ├── executor.c
│   └── builtin.c
├── include/        # Header files
│   ├── shell.h
│   ├── parser.h
│   ├── executor.h
│   └── builtin.h
├── obj/            # Object files (.o) (generated)
├── bin/            # Executable file (generated)
│   └── my_shell
├── docs/           # Documentation
│   ├── design.md
│   └── report.pdf
├── tests/          # Test programs
│   ├── test_parser.c
│   └── test_executor.c
├── scripts/        # Helper scripts
│   └── run.sh
```

```
└── assets/           # Images/screenshots
└── architecture.png
```

Note: For creating this structure use `mkdir` and `touch` commands.

```
alekhya@Alekhyareddy:~$ cd OSSP
alekhya@Alekhyareddy:~/OSSP$ tree
```

```
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
│   └── my_shell
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── fork_exec.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c
```

```
9 directories, 20 files
```

2. To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.

Task-1: Using a manual command understand the fork() and exec() system calls

Task-2: Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

Code :

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>
int main()
{
    pid_t pid;
    char command[100];
    printf("Enter a Linux command: ");
    scanf("%s", command);
    pid = fork();
    if(pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
        printf("Executing command: %s\n\n", command);
        execlp(command, command, NULL);
        printf("Command execution failed.\n");
    }
    else if(pid > 0)
```

```
{  
    printf("\nParent Process\n");  
    printf("Parent PID : %d\n", getpid());  
    printf("Child PID : %d\n", pid);  
    wait(NULL);  
    printf("\nChild process completed.\n");  
}  
else  
{  
    printf("Fork failed!\n");  
}  
return 0;  
}
```

```
alekhya@Alekhyareddy: ~  
GNU nano 7.2  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/types.h>  
  
int main()  
{  
    pid_t pid;  
  
    pid = fork();    // Create a new process  
  
    if (pid < 0)  
    {  
        printf("Fork failed!\n");  
    }  
    else if (pid == 0)  
    {  
        // Child Process  
        printf("This is the Child Process.\n");  
        printf("Child PID = %d\n", getpid());  
    }  
    else  
    {  
        // Parent Process  
        printf("This is the Parent Process.\n");  
        printf("Parent PID = %d\n", getpid());  
        printf("Child PID = %d\n", pid);  
    }  
  
    return 0;  
}
```

```
alekhya@Alekhyareddy: ~  
alekhya@Alekhyareddy:~$ ls  
LICENSE  OSSP  assets  command_executor.c  fork_exec.c  hello.c  nano.9019.save  process.c  tests  
Makefile  README.md  bin  docs  fork_program  include  obj  scripts  
OSLab  Week2  command_executor  fork_exec  hello  main.c  process  src  
alekhya@Alekhyareddy:~$ gcc fork_exec.c -o fork_exec  
alekhya@Alekhyareddy:~$ ./fork_exec  
Enter a Linux command: ls  
  
=== Parent Process ===  
Parent PID = 9106  
Child PID = 9107  
  
=== Child Process ===  
Child PID = 9107  
LICENSE  OSSP  assets  command_executor.c  fork_exec.c  hello.c  nano.9019.save  process.c  tests  
Makefile  README.md  bin  docs  fork_program  include  obj  scripts  
OSLab  Week2  command_executor  fork_exec  hello  main.c  process  src  
  
Child process completed.  
alekhya@Alekhyareddy:~$
```